DAY-23

PYTHON 90 DAYS CHALLENGES

Example 1

```
PYTHON CODE
class Student:
    pass

student1 = Student()
print(type(student1))
```

Output: <class '__main__.Student'>

Class Naming Style

Good Class Name	Reason
Student	Clear class name
BankAccount	Uses PascalCase
EmployeeRecord	Meaningful name

14.1.2 Objects

An **object** is an instance created from a class.

Syntax

object_name = ClassName()

1. An object is created from a class.
2. One class can create many objects.
3. Each object can have its own data.
4. Objects are also called instances.

Example 1

```
PYTHON CODE
class Student:
    pass
student1 = Student()
student2 = Student()
print(student1)
print(student2)
```

CodeWithNishchal

Output:
<__main__.Student object at ...>
<__main__.Student object at ...>
The memory address may be different in every run.

Class vs Object

Point	Class	Object
Meaning	Blueprint	Real instance
Created using	class keyword	Class name with ()
Example	Student	student1 = Student()
Memory	No object data yet	Stores actual object data

14.1.3 Attributes

Attributes are variables that belong to an object or class.

Syntax

object_name.attribute_name = value

Explanation

1. Attributes store data about an object.
2. Each object can have different attribute values.
3. Attributes are accessed using dot . notation.

Example 1

PYTHON CODE

```
class Student:
    pass


student1 = Student()
student1.name = "Aman"
student1.age = 21
print(student1.name)
print(student1.age)
```

Output: Aman
21

devWithNishchal

Attribute Table

Code	Meaning
student1.name	Accesses name attribute
student1.age	Accesses age attribute
student1.name = "Aman"	Creates/updates attribute

14.1.4 Methods

A **method** is a function defined inside a class.

Syntax

```python
class ClassName:
    def method_name(self):
        statement
```

1. Methods define object behavior.
2. Methods are functions inside a class.
3. Instance methods usually take self as the first parameter.
4. Methods are called using an object.

Example 1

```
PYTHON CODE
class Student:
    def greet(self):
        print("Hello")

student1 = Student()
student1.greet()
```

Output: Hello

14.1.5 self

deWithNishchal

self refers to the current object.

Syntax

```
def method_name(self):
    statement
```

1. self represents the object that is calling the method.
2. It is used to access object attributes and methods.
3. Python automatically passes the object as self.
4. self is not a keyword, but it is the standard convention.
5. Always use self for instance methods.

Example 1

PYTHON CODE

```
class Student:
    def show_name(self):
        print(self.name)

student1 = Student()
student1.name = "Aman"
student1.show_name()
```

Output: Aman

14.1.6 __init__

__init__ is a special method used to initialize object data.

Syntax

```
Python

class ClassName:
    def __init__(self):
        statement
```

1. __init__ runs automatically when an object is created.
2. It is used to set initial attribute values.
3. It is commonly called a constructor.
4. Technically, __init__ initializes the object after it is created.
5. The actual object creation is handled by __new__, which is advanced and usually not needed in core notes.

Example 1

CodeWithNishchal

```
PYTHON CODE
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Aman", 21)
print(student1.name)
print(student1.age)
```

Output: Aman
21

14.1.7 Instance Variables

Instance variables are variables that belong to a specific object.

Syntax

self.variable_name = value

1. Instance variables are usually created inside __init__.
2. Each object gets its own copy.
3. Changing one object's instance variable does not affect another object.
4. They are accessed using object.variable_name.

Example 1

```
PYTHON CODE
class Student:
    def __init__(self, name):
        self.name = name


student1 = Student("Aman")
student2 = Student("Riya")
print(student1.name)
print(student2.name)
```

Output: Aman
Riya

NOTE: student1.name and student2.name are separate instance variables.

14.1.8 Class Variables

WithNishchal

Class variables are variables shared by all objects of a class.

Syntax

```python
class ClassName:
    class_variable = value
```

Explanation

1. Class variables are defined directly inside the class.
2. They are shared by all objects.
3. They are useful for common data.
4. They can be accessed using the class name or object name.
5. Prefer accessing class variables using the class name.

Example 1

```
PYTHON CODE
class Student:
    school_name = "ABC School"

    def __init__(self, name):
        self.name = name

student1 = Student("Aman")
student2 = Student("Riya")
print(student1.school_name)
print(student2.school_name)
print(Student.school_name)
```

Output:
ABC School
ABC School
ABC School

Instance Variable vs Class Variable

hNishchal

Point	Instance Variable	Class Variable
Belongs to	Object	Class
Created where	Usually inside __init__	Inside class, outside methods
Shared?	No	Yes
Access	self.name	ClassName.variable
Example	self.name	school_name

Important Warning

Avoid mutable class variables unless shared data is intentional.

```python
class Student:
    subjects = []
```

This list is shared by all objects, which can cause unexpected behavior.

14.1.9 Methods vs Functions

A function is independent.
A method belongs to a class or object.

Comparison Table

Point	Function	Method
Defined where?	Outside class	Inside class
Called using	Function name	Object or class
First parameter	No automatic self	Instance method gets self
Example	greet()	student.greet()

14.2 OOPS Principles

14.2.1 Encapsulation

Code With Nishchal